Step 0 — Split train / dev / test e protocollo di valutazione

Obiettivo

Congelare, **prima** di qualsiasi misura o generazione di dati, gli insiemi su cui si addestra (train), si decide (dev) e si giudica una volta sola (test). Il protocollo è lo stesso di ../fine_tuning/00b_split_e_protocollo.md: qui si riassume e si indica cosa cambia per il compito unico.

Gli split possono essere **condivisi** fra i due piani: se `data/finetune/splits/` è già stato prodotto per il piano generale, si riusa identico (stesso `manifest.json`), così i risultati dei due piani sono confrontabili.

1. I blocchi

Blocco	Contenuto	Uso	Quando si guarda
dev_select	~60% del dev di ciascuna fonte di training (fino a 600 righe, bilanciate)	soglia, temperatura, checkpoint, scelta fra esperimenti	sempre
dev_confirm	~40% del dev, documenti diversi (fino a 400 righe)	conferma delle scelte	una volta per fase
test_id	test delle fonti di training (fino a 2.000 righe per fonte)	quanto il modello ha imparato	baseline e fine
test_ood	SciFact, Climate-FEVER, HealthVer/COVID-Fact, HaluBench (senza la parte RAGTruth), HalluMix, LLM-Oasis	generalizzazione	baseline e fine
AggreFact test	11 fonti, 29.320 righe	confronto con la leaderboard, **zero-shot**	baseline e fine
test_long (nuovo)	sottoinsieme di test_id e test_ood con documenti > 4.000 token	qualità del chunking (03, §4)	baseline e fine
~~Oblio~~	~~JevBench, Kev v9, typed-decisions~~	**non serve**: il modello non deve restare generale (02c)	—

2. Fonti e split

Identici al piano generale:

Fonte	train	dev (select + confirm)	test_id
FEVER	train	`paper_dev`	`paper_test`
ANLI	train R1–R3	dev R1–R3	test R1–R3
VitaminC	train	validation	test
HoVer	train	metà del dev ufficiale, per documento	altra metà
AVeriTeC	train	metà del dev ufficiale	altra metà
MiniCheck C2D/D2C	80% dei documenti	10%	10%
SummEdits	80% dei documenti	10%	10%

Fonti nuove rispetto al piano generale, solo se si usano in training (02a):

Fonte	train	dev	test_id
HotpotQA / MuSiQue (base di CG2C-MHQA)	train ufficiale	metà del dev, per domanda	altra metà
Dati generati (C2D, D2C, CG2C, `remove_evidence`, …)	**solo** dal train	—	— (eventuale "test sintetico" separato, mai mescolato con test_id)

Esclusi: RAGTruth (98% di documenti in comune con AggreFact), WiCE (solo esperimento dedicato), LLM-AggreFact (dev e test) dal training.

3. Regole contro le fughe

1. Nessun `doc_id` in più di uno split.
2. Controllo dei quasi-duplicati (13-grammi) fra train e dev/test nostri, non solo contro AggreFact.
3. Decontaminazione contro tutti i test esterni (AggreFact, test_ood).
4. **La generazione parte solo dal train**: C2D/D2C/CG2C e le coppie modificate ereditano il `doc_id` della riga di partenza.
5. **Selezione degli esempi difficili** (ANLI difficile, 02a §3): si fa con il modello di partenza **solo sul train**. Selezionare sul dev renderebbe il dev più facile o più difficile in modo non controllato.

4. Aggregazione e soglia

- BAcc per fonte → media non pesata per blocco (test_id, test_ood, AggreFact) → **numero unico = media dei tre blocchi**. AggreFact riportato anche da solo, formato leaderboard.
- **Soglia principale**: temperatura e soglia su `dev_select`, confermate su `dev_confirm`, applicate a tutti i test. Su AggreFact il risultato è zero-shot.
- **Soglia secondaria, dichiarata**: fittata sul dev di AggreFact, solo per confronto con i numeri esistenti (77,42 / 76,71 a 3 domande; 76,69 / 76,20 a una domanda).
- **Soglia fissa 0,5** (nuovo, come MiniCheck): riportata sempre. Se dopo il training la BAcc a 0,5 è entro ~0,5 punti da quella con soglia fittata, il modello è ben calibrato e in produzione si può usare 0,5 senza dipendere da un dev.

5. Output e congelamento

```
data/finetune/splits/
  train/<fonte>.jsonl
  dev_select/<fonte>.jsonl
  dev_confirm/<fonte>.jsonl
  test_id/<fonte>.jsonl
  test_ood/<fonte>.jsonl
  test_long/ids.json        # nuovo: id delle righe lunghe, presi da test_id e test_ood
  manifest.json             # righe per split e fonte, SHA-256 di ogni file, regole, data
```

6. Codice

Esiste: `data_prep/build_pairs.py`, `data_prep/make_mix.py`. Da scrivere (condiviso con il piano generale): `data_prep/make_splits.py` con split a tre vie, builder test_ood, decontaminazione estesa, manifest; in più, per questo piano, il calcolo di `test_long` con il tokenizer del modello.

Criteri per passare allo step successivo

- `manifest.json` scritto; nessun `doc_id` condiviso; decontaminazione eseguita.
- `test_long` definito (numero di righe per fonte registrato).